AI Compiler Performance Engineering

Auto-Loops Books — ch09

Contents

1 Hierarchical Synchronization and Communication **1**

1.1 Sync hierarchy 2

1.2 Hardware sync pick 4

1.3 Deadlock detection 6

1.4 YiRage sync pass 8

1.5 Key Takeaways 10

1.6 Conclusion 11

Bibliography **12**

Index **14**

Chapter 1

Hierarchical Synchronization and Communication

The double-buffer pipelines of Chapter 8 only work if the producer and consumer agree on when a buffer is ready, and a megakernel that fuses a whole layer must coordinate hundreds of units without ever returning to the host. Synchronization is therefore not a detail at the bottom of the stack—it is the mechanism that makes static roles and overlapped movement safe, and getting it wrong is how a fused kernel either corrupts data or deadlocks (Shah *et al.*, 2024; NVIDIA, 2022). The decode stakes are unchanged: dense Llama-3.2-1B issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305, and the whole point of collapsing those launches is to keep coordination on chip rather than paying a host round trip per operator (Vellaisamy *et al.*, 2026; Chen, 2026). The mistake to avoid is using a coarser synchronization scope than the dependency requires. A barrier that synchronizes a whole thread block when only two warps need to agree wastes cycles serializing units that could have run free; conversely, omitting a needed barrier is a correctness bug that surfaces as nondeterministic output (Dao *et al.*, 2022; NVIDIA, 2024). This chapter develops the synchronization hierarchy, shows how to pick the right primitive per hardware class, treats deadlock as a first-class hazard, and ends with the YiRage pass that inserts and checks synchronization per target.



Figure 1.1: The synchronization hierarchy: cost rises with scope, from a warp barrier to a multi-GPU collective. Synchronize at the narrowest scope a dependency requires.

1.1 Sync hierarchy

Synchronization on a GPU is hierarchical, mirroring the thread hierarchy of Chapter 7, and the central design rule is to **sync** at the narrowest scope that the dependency actually spans (Shah *et al.*, 2024). The reason a hierarchy exists at all is that coordination has a physical cost proportional to how far apart the units being coordinated are and how many of them there are, so a single universal synchronization primitive would be either too weak (unable to coordinate distant units) or too expensive (paying chip-crossing cost for a local handoff) (NVIDIA, 2022; Davies *et al.*, 2025). By exposing a ladder of scopes, the hardware lets software pay only for the coordination it needs.

The finest scope is the warp: `__syncwarp` coordinates the 32 threads of a warp and is nearly free because they already execute in lockstep. The next is the thread block: `bar.sync` (exposed as `__syncthreads`) is a **barrier** that all threads of a CTA must reach before any proceeds, and it is the workhorse of shared-memory kernels (NVIDIA, 2022). Between these sits the transaction **barrier** (mbarrier) of Chapter 8, which producer and consumer warps use to hand off a buffer without a full block **barrier** (Shah *et al.*, 2024).

The transaction **barrier** deserves emphasis as the scope that decode kernels live in. Unlike a block `bar.sync`, which forces every warp to a common point, a transaction **barrier** lets a producer warp signal "buffer k is full" to exactly the consumer that waits on it, so the two coordinate one tile apart while the rest of the kernel runs free (Shah *et al.*, 2024). This is the synchronization that makes the double-buffer pipeline of Chapter 8 possible: without it, overlapping a load with a compute would require a block **barrier** between them that defeats the overlap. The transaction **barrier** is thus the connective tissue of the whole MegaKernel design—it is how static roles hand work to each other without a coarse stall (NVIDIA, 2022; Dao *et al.*, 2022).

Above the block, Hopper adds cluster-scope synchronization. Thread-block clusters are co-scheduled on the same GPC and can synchronize through hardware-accelerated cluster **barrier** instructions, which is what lets the distributed-shared-memory sharing of Chapter 6 be coordinated safely (NVIDIA, 2022; Shah *et al.*, 2024). Coarser still is grid-scope synchronization across all CTAs of a kernel,

Table 1.1: The GPU synchronization hierarchy; cost rises steeply with scope, so a decode kernel coordinates at the narrowest scope a dependency spans (Shah *et al.*, 2024; Davies *et al.*, 2025).

Scope	Primitive	Relative cost
Warp	`__syncwarp`	Near-free (lockstep)
Producer/consumer	Transaction barrier	Two units, one tile apart
Block (CTA)	`bar.sync`	Stall to slowest warp
Cluster	Cluster barrier	Several co-scheduled CTAs
Grid / multi-GPU	Grid sync / collective	Chip- or network-crossing

available through cooperative-groups grid **sync** but expensive because it requires all CTAs to be resident, and coarsest is multi-GPU communication through collectives such as all-reduce and all-gather, which cross the device boundary and the interconnect (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

The cost gradient across these scopes is the whole reason the hierarchy matters. A warp **barrier** is a few cycles; a block **barrier** stalls until the slowest warp arrives; a cluster **barrier** spans several CTAs; a grid **sync** or a collective crosses the chip or the network and can cost orders of magnitude more (Shah *et al.*, 2024; Davies *et al.*, 2025). For decode this gradient is acute because the per-token work is tiny, so a coarse **sync** on the critical path is overhead with nothing to amortize it against. The pingpong scheduling of Chapter 7—overlapping one warpgroup's softmax with another's GEMM—is exactly a use of narrow-scope barriers to coordinate two units without a block-wide stall, and it is why that technique recovered tens of TFLOPS (Shah *et al.*, 2024).

A useful way to see why scope dominates cost is to think about how many units a **barrier** must wait for and how far apart they are physically. A warp **barrier** waits for 32 threads that already advance together, so it is essentially a no-op that the hardware can satisfy immediately (NVIDIA, 2022). A block **barrier** waits for every warp in the CTA, so it stalls until the straggler arrives—and at batch-1 decode, where warps may have unequal work, that straggler sets the pace. A cluster **barrier** additionally crosses CTA boundaries within a GPC, and a grid **sync** requires every CTA in the kernel to be co-resident and to reach the same point, which is why it is only usable in a persistent kernel and is still expensive. A multi-GPU collective leaves the chip entirely, traversing NVLink or a network fabric, so its latency is measured in microseconds against the nanoseconds of an on-chip **barrier** (Davies *et al.*, 2025; Chen, 2026).

This gradient also explains why disaggregated decode deployments care so



Figure 1.2: Each hardware class provides different synchronization primitives; the compiler picks the one matching the dependency and the target.

much about where synchronization lands. When a model is split across devices for tensor parallelism, every layer may require an all-reduce to sum partial results, and that collective is on the per-token critical path (Davies *et al.*, 2025). The design response is the same producer-consumer overlap one scope coarser: overlap the collective with the next layer's compute so the network latency hides behind useful work, exactly as a transaction **barrier** overlaps a TMA load with a WGMMA (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). The synchronization hierarchy is thus self-similar—the same overlap principle applies at every scope, only the latency to hide grows.

1.2 Hardware sync pick

Picking the right synchronization primitive is target-specific, and the three classes differ as much in synchronization as they did in memory and movement. On a **gpu** the primitives are the barrier family above plus atomics for fine-grained updates; the design choice is which scope to use and whether to use a transaction barrier for asynchronous handoff or a full block barrier for a bulk-synchronous phase (Shah *et al.*, 2024; NVIDIA, 2022). CUTLASS wraps the producer/consumer barrier discipline in reusable asynchronous pipeline classes precisely because hand-managing a circular list of barriers across many asynchronous operations is too error-prone for a human, which is a strong hint that synchronization belongs in a compiler pass rather than in hand-written kernels (Shah *et al.*, 2024).

A subtlety specific to the **gpu** is the difference between a barrier and an atomic, and when each is the right tool. A **barrier** enforces that all participating units reach a point before any proceeds—a phase boundary—whereas an atomic enforces that a single memory update happens indivisibly, allowing units to coordinate without all stopping (NVIDIA, 2022). For a reduction where many warps contribute to one sum, an atomic add lets them proceed independently and merge results, avoiding a full **barrier**; for a phase change where the next stage cannot start until the previous fully completes, a **barrier** is required. Choosing atomics where possible keeps units running and is often the better fit for the irregular, low-parallelism reductions of batch-1 decode, while barriers remain necessary at genuine phase

boundaries (Shah *et al.*, 2024; Chen, 2026). The transaction **barrier** is a hybrid—it is a **barrier** scoped to a producer-consumer pair, giving phase-boundary semantics without a block-wide stall.

On a **cpu** synchronization is built from memory fences, atomic operations, and the cache-coherence protocol rather than explicit barriers into a scratchpad (Goodfellow *et al.*, 2016). Threads coordinate through atomic flags and acquire/release fences that order memory operations, and the hardware's coherence protocol makes a write by one core visible to another—at the cost of coherence traffic that, like NUMA, can dominate a memory-bound decode loop if threads share data across sockets. The **cpu** design choice is to minimize cross-core sharing on the hot path, partitioning work so that each thread's data is mostly private and synchronization is rare, the opposite of a GPU kernel that synchronizes warps every pipeline stage.

On an **npu** synchronization is explicit and local: AIE-ML tiles use locks and task-completion-tokens, where the DMA signals a token when a buffer is ready and the vector unit waits on it before consuming, and neighboring tiles coordinate through configured streams and cascade connections (AMD, 2024a,b). Because the array is statically configured, the synchronization pattern is fixed at compile time—there is no dynamic contention for a shared lock as there might be on a CPU—which makes the **npu** the most deterministic of the three but also the least forgiving: a mis-sequenced token is a compile-time bug that stalls the pipeline rather than a rare race. Across collectives, multi-device decode must also pick a communication pattern—all-reduce for tensor-parallel sums, all-gather for sharded weights—and overlap it with compute, which is the same producer-consumer overlap one scope coarser (Davies *et al.*, 2025; Chen, 2026).

The deeper contrast is between explicit and implicit synchronization, and it tracks the same axis as residency and movement. The **gpu** sits in the middle: barriers are explicit, but the hardware still schedules warps dynamically, so synchronization coordinates a partly dynamic machine. The **cpu** is the most implicit: the coherence protocol synchronizes caches automatically, and the programmer mostly reasons about ordering through fences rather than issuing explicit barriers, which is convenient but makes the synchronization cost hard to see and easy to incur accidentally through false sharing (Goodfellow *et al.*, 2016). The **npu** is the most explicit: every lock and token is placed by the compiler, nothing is automatic, and the cost is fully visible at compile time (AMD, 2024a,b). A residency-first engineer reads this as the same lesson once more—the more explicit the control, the more the compiler can reason about and minimize the coordination, which is why the statically scheduled **npu** is the most predictable and the coherence-driven **cpu** the most prone to hidden synchronization cost.

A concrete **cpu** hazard worth calling out is false sharing: two threads writing different variables that happen to share a cache line force the coherence protocol to ping the line between cores on every write, synchronizing data that did not logically need synchronizing (Goodfellow *et al.*, 2016). For a memory-bound decode loop this can quietly dominate, and the fix is a layout decision—pad per-thread state to separate cache lines—not a synchronization primitive. It is the **cpu** analogue of the GPU's bank conflicts: an accidental sharing that the compiler must lay out around.

1.3 Deadlock detection

Static role assignment and asynchronous pipelines introduce a hazard that synchronous, per-operator kernels largely avoid: **deadlock**. When producer and consumer units coordinate through barriers, a mismatch in their arrive/wait counts, or a circular dependency where each unit waits for the other, hangs the kernel permanently (Shah *et al.*, 2024; NVIDIA, 2022). On a GPU a **deadlock** typically manifests as a kernel that never returns; on an NPU it manifests as a pipeline stage waiting forever on a token that will never be signaled (AMD, 2024a). The insidious property is that these failures are often nondeterministic or configuration-dependent: a barrier-count assumption that holds at one block size or warp count breaks at another, so a kernel that passes tests on one shape hangs on a production shape (Shah *et al.*, 2024; NVIDIA, 2022). This is the worst kind of bug to discover in a serving system, because it surfaces as a hung request under specific load rather than as a clean failure in testing, which is the strongest argument for proving deadlock-freedom statically rather than relying on test coverage.

Because these are coordination bugs, not arithmetic bugs, they do not show up in a numerical test—the kernel simply never finishes.

The classic **deadlock** patterns are worth naming because a compiler can check for them. The first is a barrier-count mismatch: if a barrier expects n arrivals but only $n-1$ units reach it (because one took a divergent path), the barrier never releases (Shah *et al.*, 2024). The second is a circular wait: producer waits for the consumer to free a buffer while the consumer waits for the producer to fill it, a cycle that only a correct initial condition (the producer starting with empty buffers) avoids. The third is an ordering bug in a multi-stage pipeline where the barriers are not arranged as a consistent circular list, so two stages wait on each other out of order (Shah *et al.*, 2024).

There is a reason these patterns proliferate exactly when performance improves. Each optimization in this Part trades a bulk-synchronous structure—compute

everything, synchronize, move on—for a finer-grained asynchronous one where units run ahead and coordinate point-to-point (Shah *et al.*, 2024; Dao *et al.*, 2022). Bulk-synchronous code is hard to deadlock because there is one barrier everyone hits; asynchronous, warp-specialized code has many barriers with specific arrive/wait counts, and every one is an opportunity for a mismatch. In other words, **deadlock** risk is the price of the overlap that makes decode fast, which is why a serious decode kernel cannot treat synchronization casually—the very structure that wins latency is the structure that can hang.

The role of control-flow divergence in **deadlock** deserves emphasis because it is the subtlest case. A **barrier** placed inside a conditional that only some warps take will wait forever for the warps that branched around it (NVIDIA, 2022; Shah *et al.*, 2024). This is why barriers must be placed at points all participating units are guaranteed to reach, and why a compiler that understands the control flow is better positioned to verify this than a human reading a complex kernel. On a statically scheduled **npu** the analogous bug is a token that one tile expects but no tile ever produces because a stage was conditionally skipped, and again the static schedule makes it checkable at compile time (AMD, 2024a,b).

The defense is to make synchronization structural rather than ad hoc. CUTLASS's pipeline classes exist so that the arrive/wait counts and the circular barrier list are generated from a single specification, guaranteeing they match (Shah *et al.*, 2024). A compiler that owns synchronization can go further: it can verify statically that every barrier's expected arrival count equals the number of units that reach it on all paths, that the producer-consumer cycle has a correct initial condition, and that the barrier ordering forms a consistent list—turning **deadlock** from a runtime hang discovered in production into a compile-time rejection (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). On a statically scheduled NPU this check is especially natural because the entire synchronization pattern is known at compile time (AMD, 2024b). The same staticness that makes an NPU unforgiving—no cache to hide a late load, no dynamic scheduler to reorder around a stall—is what makes its synchronization fully analyzable: with every lock, token, and stream assignment fixed before execution, the compiler can simulate the coordination and prove progress, something far harder on a dynamically scheduled GPU where warp progress depends on runtime occupancy (AMD, 2024a; NVIDIA, 2022). This is a recurring trade in the book: explicit, static control is harder to write by hand but easier for a compiler to verify, which is exactly why these facets belong in a pass.

1.4 YiRage sync pass

The book's reference compiler turns synchronization into a generated, checked pass. **YiRage** carries a **sync** pass that, given a fused region with its static roles and pipeline, inserts the synchronization at the narrowest correct scope per target—warp and transaction barriers on a GPU, fences and atomics on a CPU, locks and tokens on an NPU—and verifies the result is deadlock-free before codegen (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Because the pass consumes the role assignment of Chapter 7 and the pipeline of Chapter 8, it knows exactly which units must coordinate and when, so it can choose warp-scope over block-scope wherever the dependency permits and avoid the coarse-**sync** overhead that decode cannot afford (NVIDIA, 2022; Chen, 2026).

Choosing the scope is itself an optimization the pass performs, not just a correctness step. Given the dependency graph of a fused region, the pass determines, for each handoff, the smallest set of units that actually share data and synchronizes only them—two warps through a transaction **barrier** rather than the whole block, or a cluster **barrier** only where distributed shared memory is genuinely used (Shah *et al.*, 2024; NVIDIA, 2022). This matters because a human writing a kernel tends to reach for the familiar block `bar.sync` even when a narrower primitive would do, leaving performance on the table; the pass has no such habit and picks the minimal scope mechanically. For batch-1 decode, where every coarse **sync** on the critical path is unamortized overhead, this minimization is a direct latency win (Vellaisamy *et al.*, 2026; Chen, 2026).

The pass also adapts the synchronization to the precision and the pipeline depth, because those change what must be coordinated. A deeper pipeline has more buffers and therefore more transaction barriers in flight, and the pass must generate a consistent circular list of exactly the right length (Shah *et al.*, 2024). A lower-precision format that packs more elements per buffer can change which units produce and consume a given tile, shifting where the barriers belong. Because the **sync** pass runs after the tiling and pipeline passes have fixed these decisions, it has the information to place the barriers correctly, which is the argument for synchronization being a late, dependent pass rather than something hand-written up front (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The pass's verification is the part that matters most for a megakernel. A whole-model megakernel coordinates hundreds of units, and a single mismatched barrier hangs the entire forward pass, so shipping one without a static deadlock check is reckless (Mirage Project contributors, 2025; Wu *et al.*, 2024). The **YiRage sync** pass checks barrier arrival counts against the units on every control-flow

path, validates the producer-consumer initial conditions, and confirms the barrier ordering forms a consistent list, rejecting any region that cannot be proven deadlock-free (Chen and YiRage Contributors, 2026). This composes with the residency, role, and pipeline checks of the previous chapters: the superoptimizer proposes a fusion, the memory pass certifies its footprint, the tiling pass assigns roles, the pipeline pass overlaps movement, and the **sync** pass guarantees the coordination is correct and minimal—all per target, all before a kernel is emitted (Wu *et al.*, 2024; Mirage Project contributors, 2025).

The verification also has to reason about the multi-device boundary when a model is sharded. A tensor-parallel layer's all-reduce is a synchronization point across devices, and a mismatch there—one rank expecting a collective that another never issues—hangs the whole job exactly like a barrier-count mismatch within a kernel, but now across the network (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The **sync** pass treats the collective as another synchronization edge in the dependency graph, checking that every participating rank issues a matching collective and that the overlap with compute does not create a cycle where a rank waits to send while a peer waits to receive. This unifies on-chip and cross-device coordination under one analysis, which matters because production decode increasingly spans many devices and a cross-device **deadlock** is even harder to debug by hand than an on-chip one (Davies *et al.*, 2025; Chen, 2026).

A final benefit of owning synchronization in the compiler is reproducibility across targets. A hand-written kernel's synchronization is tuned for one GPU and silently wrong or suboptimal on another—a barrier count assuming a specific warp count, say, or an atomic pattern that contends differently under another memory model (NVIDIA, 2022; AMD, 2024b). Because the **YiRage sync** pass regenerates the synchronization from the dependency graph per target, the same logical kernel gets a correct plan on each, with the scope and primitive chosen for that machine (Chen and YiRage Contributors, 2026). For a fleet that mixes GPU generations, CPUs, and NPUs, this is the difference between maintaining one verified specification and maintaining a matrix of hand-tuned, separately-debugged kernels.

The result is that synchronization, the most error-prone part of a hand-written fused kernel, becomes a property the compiler generates and proves rather than one a CUDA expert debugs by staring at hangs. For a serving team deploying megakernels across a heterogeneous fleet, that guarantee is what makes end-to-end fusion safe: the same logical kernel gets a different, minimal, deadlock-free synchronization plan on an H100, a CPU, and an XDNA tile, each checked before it runs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

It is worth situating this within the book's recurring argument. Chapters 6 through 9 have each taken one facet of a fused decode kernel—residency, role assignment, asynchronous movement, and now synchronization—and shown that the facet is best handled as a checked, per-target compiler pass rather than as hand-written CUDA, because each is a place where prefill intuitions mislead and where a single hand-tuned artifact fails to port (Vellaisamy *et al.*, 2026; Chen, 2026). Synchronization completes the set: it is the facet that turns the other three from independent optimizations into a single correct kernel, because residency without coordination races, roles without coordination deadlock, and pipelines without coordination read stale buffers (Shah *et al.*, 2024; AMD, 2024a). Treating all four as composable passes over a shared representation is what lets a superoptimizer propose an aggressive fusion and have the compiler certify it is resident, role-assigned, overlapped, and deadlock-free before a single token is decoded (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

1.5 Key Takeaways

- **Synchronize at the narrowest correct scope.** Cost rises steeply from a warp barrier to a block barrier to a cluster barrier to a grid sync or multi-GPU collective, so use the narrowest scope a dependency spans (Shah *et al.*, 2024; Davies *et al.*, 2025). A coarse sync on the decode critical path is pure overhead with nothing to amortize it against (Vellaisamy *et al.*, 2026).

- **Pick the primitive per hardware class.** GPUs use barriers, transaction barriers, and atomics; CPUs use fences, atomics, and cache coherence; NPUs use locks and task-completion-tokens; multi-device uses collectives (NVIDIA, 2022; AMD, 2024a). Minimize cross-core sharing and false sharing on a CPU; the NPU pattern is fixed at compile time (AMD, 2024b).

- **Treat deadlock as a first-class hazard.** Barrier-count mismatches, circular waits, and inconsistent barrier ordering hang a fused kernel without any numerical error (Shah *et al.*, 2024). They are coordination bugs invisible to a correctness test and often shape- or configuration-dependent, so they hang in production rather than in testing (NVIDIA, 2022).

- **Make synchronization structural and checkable.** Generate arrive/wait counts and the circular barrier list from one specification, as CUTLASS pipeline classes do, so they cannot mismatch (Shah *et al.*, 2024). A compiler can verify deadlock-freedom statically (Wu *et al.*, 2024).

- **Generate and prove sync per target.** The YiRage sync pass inserts minimal, correct synchronization per class and rejects any region not provably deadlock-free, composing with the residency, role, and pipeline passes (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Megakernels coordinate hundreds of units, so the proof is essential (Chen, 2026).

1.6 Conclusion

Synchronization is what makes the static roles of Chapter 7 and the asynchronous pipelines of Chapter 8 safe: a hierarchy of scopes from warp barriers to multi-GPU collectives, with cost rising sharply as scope widens, so the discipline is to coordinate at the narrowest scope a dependency requires (Shah *et al.*, 2024; Davies *et al.*, 2025). Each hardware class supplies different primitives—GPU barriers, CPU fences, NPU tokens—and each introduces deadlock as a coordination hazard invisible to numerical tests, which is why the YiRage sync pass inserts minimal synchronization per target and proves it deadlock-free before codegen (Chen and YiRage Contributors, 2026; AMD, 2024a). With residency, roles, movement, and synchronization all now compiler-managed and checked, the MegaKernel machinery is complete enough to tackle the algorithm that ties them together in the decode hot path: Chapter **??** develops the online-softmax attention optimization that keeps the running statistics resident and synchronized across the very pipeline stages this chapter coordinated (Dao *et al.*, 2022; Chen, 2026).

Bibliography

AMD (2024a). Ai engine technology. https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html. 5, 6, 7, 10, 11

AMD (2024b). AMD XDNA architecture. https://www.AMD.com/en/technologies/XDNA.html. 5, 7, 9, 10

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 3, 5, 8, 9, 10, 11

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. https://github.com/chenxingqiang/YiRage. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 7, 8, 9, 10, 11

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 7, 11

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 2, 3, 4, 5, 9, 10, 11

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 3, 4, 5, 6, 9

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. https://github.com/mirage-project/mirage. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 8, 9, 11

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 4, 6, 7, 8, 9, 10

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMA), persistent kernels with tile scheduler, pingpong scheduling. 1, 2, 3, 4, 5, 6, 7, 8, 10, 11

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 8, 10

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 7, 8, 9, 10

Index

Barrier, 1

Deadlock, 1

Synchronization, 1